

Aplikasi Matriks Tindak Lanjut

Document Design Architecture

Revision 1.0

Juli 2025

Created by

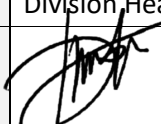


Supported by:

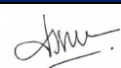
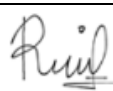
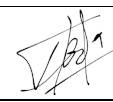


I. APPROVAL

Prepared By	
Company	PT Sinergi Mitra Lestari Indonesia (SINERGI ID)
Full Name	Mastuki
Designation	Application & Solution Lead Manager
Signature	
Date	01/08/2025

Approved By	
Company	PT Mineral Industri Indonesia (Persero) ("MIND ID")
Full Name	Arini Kasmira
Designation	Portfolio Management Division Head
Signature	
Date	11/08/2025

II. ACKNOWLEDGE

	Full Name	Company	Designation	Date	Signature
1	Janwar Iskandar	SINERGI ID	Project Leader	25/07/2025	
2	Riyandi Firman Pratama	SINERGI ID	Business Analyst	25/07/2025	
3	Mifta Adiwira Jati Jaka	SINERGI ID	ICT Business Analyst	01/08/2025	
4	AH Wirayudha	SINERGI ID	ICT Software Engineer	01/08/2025	
5	Reyhan Zacky Rivai	MIND ID	Staff Departemen Subsidiary Management	11/08/2025	
6	Hendra Ferdila	MIND ID	Subsidiary Management Department Head	11/08/2025	
7	Rudy Hendrayanto	MIND ID	IT Software Engineer	07/08/2025	
8	Annisa Latasha	MIND ID	ICT Data Analytics Senior Associate	07/08/2025	
9	Mochamad Radityo	MIND ID	Plh Department Head ICT Strategy, Architecture ERP & Digitalization	07/08/2025	
10					

Project Identification

Project	
Project Name	Pengadaan Aplikasi Matriks Tindak Lanjut
Client Name	PT Mineral Industri Indonesia (Persero) ("MIND ID")
Project Manager	Mastuki (PT Sinergi Mitra Lestari Indonesia (Sinergi ID) – PIC)

Revision History

Author	Date Issued	Version	Comments

Table of Contents

1.	Pendahuluan	5
2.	Tujuan Dokumen.....	5
3.	Deskripsi Aplikasi.....	5
4.	Arsitektur Sistem	7
5.	Detail Komponen	13
6.	Security & Authorization.....	18
7.	Strategi Deployment dan Skalabilitas	19
8.	Kesimpulan.....	20

1. Pendahuluan

Aplikasi Matriks Tindak Lanjut merupakan sistem yang akan mendukung upaya implementasi fungsi MIND ID sebagai “Strategic Active Management” untuk dapat melakukan pendampingan secara aktif dan melakukan pemantauan dari arahan-arahan MIND ID yang disampaikan kepada Anggota MIND ID pada pelaksanaan Rapat Monitoring & Evaluation (Rapat MONEV), dengan sistem ini, MIND ID bisa lebih mudah mengumpulkan, menilai, dan memvalidasi tindak lanjut dari setiap arahan, sehingga proses monitoring menjadi lebih efisien dan terpantau dengan jelas.

2. Tujuan Dokumen

Dokumen ini menjelaskan arsitektur teknis Aplikasi Matriks Tindak Lanjut. Tujuannya adalah untuk menyediakan panduan dalam pengembangan aplikasi dan memastikan bahwa semua pihak memahami kerangka dan komponen yang terlibat.

3. Deskripsi Aplikasi

Aplikasi Matriks Tindak Lanjut adalah aplikasi yang digunakan untuk mendukung proses monitoring kinerja anggota holding terhadap tindak lanjut isu-isu strategis yang menjadi arahan MONEV, juga dapat memastikan proses komunikasi dan koordinasi antara MIND ID dan Anggota MIND ID berjalan dengan lancar dan efisien, serta pengelolaan supervisi dan evaluasi terhadap proses tindak lanjut arahan MONEV Anggota MIND ID lebih terdokumentasi dengan baik.

Berikut detail modul Aplikasi Matriks Tindak Lanjut yang akan dikembangkan:

1. Login
 - a. Multi Factor Authentication (MFA)
 - b. Single Sign On (SSO)
 - c. SignUp
 - d. Forgot Password
 - e. User Verification
2. Dashboard
 - a. Daftar Arahan & Sub Arahan
 - b. Duedate Arahan & Sub Arahan
 - c. Status Arahan & Sub Arahan

- d. Rekap Jumlah Arahan & Sub Arahan
 - e. Daftar Tindak Lanjut Arahan
 - f. Progress Tindak Lanjut Arahan
 - g. Rekap Tindak Lanjut Arahan
 - h. Notification System
- 3. Task List
 - a. Task List Monitoring
 - b. Eksport to Excel
- 4. Matriks Tindak Lanjut
 - a. Setting Arahan
 - Daftar Setting Arahan
 - Upload & Download Template
 - Export Excel
 - Upload Bulk
 - Form Arahan Detail
 - Re Open (Create tasklist & Workflow Re-Submit)
 - b. Review Arahan (Counterpart/Divisi MIND ID)
 - c. Penetapan Arahan
 - d. Tindak Lanjut Arahan
 - e. Review Tindak Lanjut
 - f. Finalisasi Status Arahan
- 5. Master Data
 - a. Divisi
 - b. Anggota Holding
 - c. Topik
 - d. Tipe Arahan
 - e. General Parameter
- 6. User Management
 - a. User Directory
 - b. Modules
 - c. Role & Priviledges

7. Reporting

- a. Daftar Arahan & Sub Arahan (Include Export excel, pdf)
- b. Tindak Lanjut Arahan

4. Arsitektur Sistem

4.1. Arsitektur Source Code

Arsitektur pada source code Matriks Tindak Lanjut ini mengikuti prinsip Clean Architecture, dirancang untuk membantu membangun aplikasi yang kuat dan mudah dipelihara. Clean Architecture mendorong pemisahan yang jelas antara berbagai tanggung jawab, membuat kode Anda lebih terorganisir, dapat diuji, dan mudah disesuaikan.

1. Backend

Backend pada aplikasi Matriks Tindak Lanjut ini menggunakan Laravel versi terbaru yaitu versi 11. Untuk arsitektur code mengikuti best practice yang umum digunakan diantaranya:

a. Migration

Migration adalah alat yang digunakan untuk mengelola skema basis data di Laravel. Fungsinya mirip dengan kontrol versi untuk basis data, memungkinkan perubahan pada tabel dan struktur basis data (seperti membuat, mengubah, atau menghapus tabel) tanpa harus secara langsung menulis SQL. Migration memudahkan pengelolaan database dalam tim dan memastikan lingkungan pengembangan sinkron.

b. Model

Model adalah representasi dari tabel dalam basis data. Setiap model biasanya terkait dengan satu tabel dalam database, dan digunakan untuk berinteraksi dengan data, termasuk mengambil, menyimpan, memperbarui, atau menghapus data dari tabel tersebut.

c. Repository

Repository adalah pola desain yang bertujuan untuk memisahkan logika akses data dari lapisan logika bisnis. Dalam konteks Laravel, repository menyimpan operasi CRUD dan abstraksi database dari controller atau model. Repository memberikan

cara yang lebih bersih dan terstruktur untuk mengelola data dan memungkinkan pengembangan yang lebih mudah di-maintain dan di-test.

d. Controller

Controller di Laravel bertanggung jawab untuk menangani permintaan HTTP dan mengembalikan respons ke pengguna. Controller menghubungkan view, model, dan logika aplikasi. Ini adalah tempat untuk mengelola aliran data dari user input, menghubungi model untuk mendapatkan data yang sesuai, dan menentukan bagaimana data tersebut akan disajikan kepada pengguna.

e. FormRequest

FormRequest adalah kelas khusus di Laravel untuk menangani validasi form. Dengan menggunakan FormRequest, validasi form bisa dipisahkan dari controller, membuat kode lebih bersih dan terorganisir. Selain itu, FormRequest juga memungkinkan penanganan aturan validasi yang lebih kompleks dan custom.

f. JsonResponse

JsonResource adalah komponen Laravel yang digunakan untuk mengubah data model menjadi representasi JSON yang lebih terstruktur. Ini sangat berguna saat membangun API, karena memungkinkan pengelolaan dan format respons data menjadi lebih teratur dan konsisten.

g. Route

Route di Laravel adalah mekanisme yang menghubungkan permintaan URL ke logika aplikasi, seperti controller atau closure. Setiap route mendefinisikan bagaimana permintaan HTTP (GET, POST, PUT, DELETE, dll.) dipetakan ke tindakan yang tepat dalam aplikasi.

2. Frontend

Backend pada aplikasi Matriks Tindak Lanjut ini menggunakan React versi terbaru yaitu versi 18. Untuk arsitektur code mengikuti best practice yang umum digunakan diantaranya:

a. Redux-Saga

Redux-Saga adalah middleware untuk Redux yang memungkinkan penanganan efek samping (seperti pemanggilan API, akses data asinkron, dan penanganan side-

effects lainnya) dalam aplikasi React. Redux-Saga bekerja dengan konsep "generator functions" untuk menjalankan tugas secara asinkron.

b. Modules

Dalam konteks React, "modules" biasanya mengacu pada cara memisahkan kode menjadi bagian-bagian yang lebih kecil dan terpisah, seperti file atau direktori, yang masing-masing menangani aspek tertentu dari aplikasi. Pendekatan modular ini memudahkan pemeliharaan, pengujian, dan pengembangan aplikasi.

c. QTable

QTable adalah komponen yang dirancang untuk menampilkan data dalam format tabel, tetapi dengan fungsi yang disesuaikan dengan kebutuhan aplikasi. Biasanya, komponen ini dibuat untuk mempermudah menampilkan data secara dinamis dan memungkinkan fitur tambahan seperti sorting, filtering, pagination, dan lain-lain.

d. Components

Komponen adalah elemen dasar dari aplikasi React. Setiap komponen adalah bagian UI yang dapat berdiri sendiri dan dapat digunakan kembali di tempat lain. Komponen bisa berupa fungsi (function components) atau kelas (class components) dan menerima props sebagai input, lalu mengembalikan elemen React sebagai output.

e. Routes

Routes mengacu pada konfigurasi rute yang menentukan bagaimana URL tertentu dipetakan ke komponen yang memungkinkan navigasi antara berbagai halaman atau komponen dalam aplikasi single-page (SPA).

4.2. Komponen Arsitektur

1. Frontend (PresentationLayer)

React Redux Pattern adalah pola arsitektur yang digunakan untuk mengelola state secara global dalam aplikasi React. Dengan menggunakan Redux sebagai pengelola state, React dapat berinteraksi dengan state secara terpusat, sehingga alur data menjadi lebih terstruktur dan predictable mengikuti prinsip-prinsip flux pattern, di mana aliran data berjalan secara satu arah, yang memudahkan pengelolaan state dalam aplikasi.

2. Backend (Business Logic Layer)

Arsitektur backend menggunakan Laravel sebagai framework utama dengan pola desain MVC (Model-View-Controller), namun komponen View tidak digunakan secara penuh karena antarmuka pengguna (frontend) dibangun terpisah menggunakan React. Dalam konteks ini, Laravel berperan sebagai RESTful API server yang hanya menangani logika bisnis dan pengelolaan data, sementara React menangani seluruh tampilan UI.

Berikut penyesuaian masing-masing komponen dalam konteks Laravel sebagai backend API:

a. Model

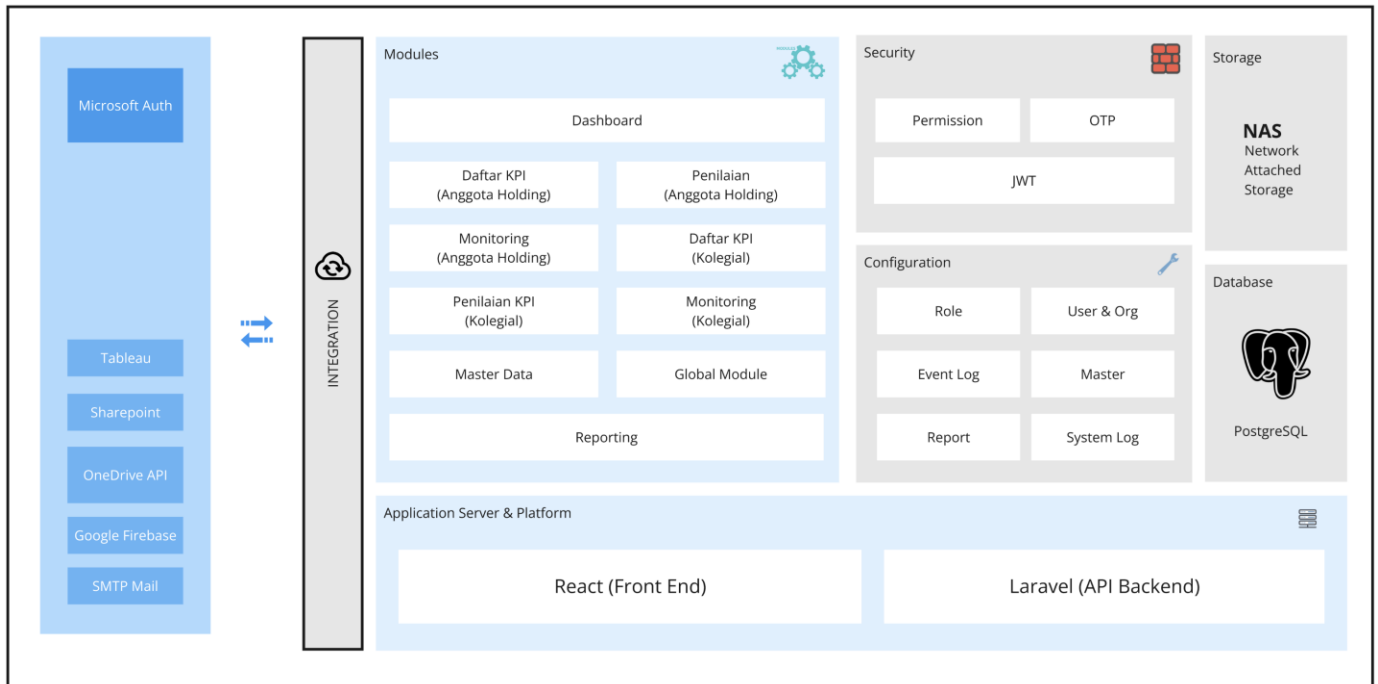
Model bertanggung jawab untuk menangani logika bisnis dan interaksi dengan database. Dalam Laravel, model digunakan untuk mewakili entitas dari database dan bekerja dengan Eloquent ORM (Object-Relational Mapping), yang memungkinkan pengembang berinteraksi dengan database menggunakan objek PHP alih-alih menulis query SQL secara langsung.

b. Controller

Controller bertindak sebagai perantara antara client (dalam hal ini, React frontend) dan model. Controller menerima permintaan HTTP (biasanya dalam bentuk JSON), memproses data atau memanggil model sesuai kebutuhan, lalu mengembalikan respon berupa JSON API. Controller tidak lagi mengembalikan view Blade, tetapi berfungsi sebagai endpoint untuk komunikasi data antara frontend dan backend.

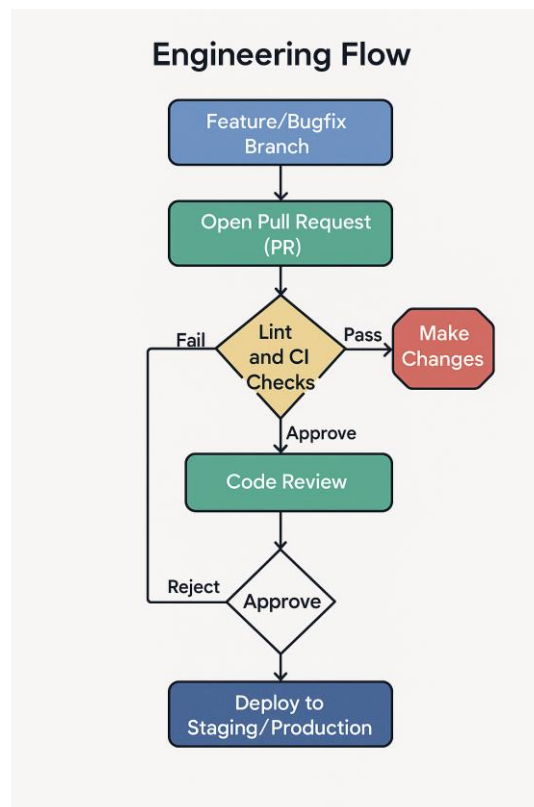
4.3. Diagram Arsitektur

Aplikasi Matriks Tindak Lanjut menggunakan sistem arsitektur Monolith. Arsitektur monolith pada aplikasi Laravel adalah pendekatan di mana seluruh aplikasi dibangun sebagai satu unit tunggal, mencakup semua komponen seperti antarmuka pengguna (UI), logika bisnis, akses data, dan lapisan lainnya dalam satu kode base yang terintegrasi. Ini adalah pendekatan umum untuk aplikasi skala kecil hingga menengah, atau saat pengembangan awal, karena kesederhanaan dalam pengelolaan dan deployment.



4.4. Engineering Flow – Quality Control Codebase

Untuk menjaga kualitas kode, stabilitas sistem, dan kolaborasi tim, sistem ini mengadopsi alur pengembangan terstruktur berikut:



a. Branching Strategy (Git)

- Main/master: Branch utama untuk deployment ke production.
- Develop: Branch untuk penggabungan fitur sebelum dirilis.
- Feature/: Branch untuk pengembangan satu fitur spesifik. Misal: feature/review-notifikasi
- Bugfix/: Branch untuk perbaikan bug.
- Hotfix/: Digunakan untuk patch mendesak di production.

b. Pull Request (PR) Rule

Setiap merge ke develop atau main harus melalui Pull Request (PR) yang mematuhi aturan berikut:

1. Minimum Reviewer: Minimal 1–2 rekan tim melakukan review dan approval.
2. Linting Passed: PR hanya bisa di-merge jika pengecekan lint bebas dari kesalahan.
3. CI/CD Check Passed: Semua unit test dan integration test harus lolos.
4. No Direct Push to Main: Hanya PR yang boleh merge ke main.

c. Code Linting

Linting digunakan untuk menjaga konsistensi penulisan kode (coding style) dan mendeteksi potensi error sejak awal.

- Tool: PHP-CS-Fixer atau PHP_CodeSniffer
- Enforcement:
 - Dijalankan secara lokal sebelum commit (pre-commit hook)
 - Dijalankan kembali di pipeline CI/CD saat PR

d. Continuous Integration / Continuous Development (CI/CD)

CI/CD adalah rekayasa perangkat lunak yang bertujuan untuk meningkatkan kualitas kode, mempercepat siklus rilis, serta meminimalkan risiko kesalahan saat penggabungan dan penyebaran aplikasi.

Continuous Integration (CI) memastikan bahwa setiap perubahan kode yang dikirimkan ke repositori secara otomatis diuji dan divalidasi melalui pipeline. Hal ini membantu mendeteksi bug lebih awal, menjaga stabilitas sistem, dan mempercepat proses pengembangan.

Continuous Deployment (CD) melanjutkan proses tersebut dengan melakukan deployment otomatis ke lingkungan tertentu, seperti staging atau production, setelah perubahan dinyatakan lolos dari tahap pengujian.

Implementasi CI/CD pada sistem ini dilakukan dengan mekanisme sebagai berikut:

- Pipeline CI secara otomatis dijalankan setiap kali terdapat commit atau pull request (PR) baru.
- Deployment ke lingkungan staging dilakukan secara otomatis setelah terjadi merge ke branch develop.
- Deployment ke lingkungan production dilakukan secara manual oleh administrator setelah PR ke branch main disetujui.
- Seluruh proses deployment terdokumentasi secara sistematis melalui pencatatan log untuk keperluan audit dan pelacakan.

5. Detail Komponen

5.1. Frontend (Presentation Layer)

- Fungsi : Memberikan antarmuka pengguna untuk aplikasi.
- Teknologi :

a. React

React adalah library JavaScript yang digunakan untuk membangun antarmuka pengguna (UI). React berfokus pada pembuatan komponen UI yang dapat digunakan kembali dan bekerja berdasarkan konsep Virtual DOM untuk mempercepat pembaruan UI secara efisien.

b. Redux

Redux adalah library pengelolaan state global untuk aplikasi JavaScript, terutama sering digunakan dengan React. Redux membantu mengelola state aplikasi yang kompleks dengan menyimpan state di satu tempat terpusat, memungkinkan setiap komponen di aplikasi dapat mengakses state tersebut.

c. Redux-Saga

Redux-Saga adalah middleware untuk Redux yang menangani efek samping asinkron seperti pemanggilan API atau delay. Redux-Saga menggunakan

generator functions untuk menangani tugas-tugas ini dan memungkinkan penanganan logika yang lebih terstruktur.

d. Axios

Axios adalah library JavaScript yang digunakan untuk melakukan permintaan HTTP, seperti GET, POST, PUT, dan DELETE. Axios sering digunakan di React untuk mengambil data dari API atau mengirim data ke server.

e. Bootstrap

Bootstrap adalah framework CSS yang membantu pengembangan front-end dengan menyediakan komponen UI yang siap pakai, seperti grid system, tombol, navigasi, dan form. Bootstrap memungkinkan pengembang membuat aplikasi yang responsif dan menarik dengan lebih cepat tanpa harus menulis banyak CSS.

f. Vite

Vite adalah build tool yang modern untuk proyek-proyek frontend, khususnya yang menggunakan React, Vue, atau framework modern lainnya. Vite dirancang untuk memberikan pengembangan yang sangat cepat dengan hot module replacement (HMR) dan bundling yang efisien saat production.

5.2. Backend (Business Logic Layer)

- Fungsi : Memproses logika aplikasi dan mengatur komunikasi antara frontend dan database.
- Teknologi :

a. PHP

PHP (Hypertext Preprocessor) adalah bahasa pemrograman server-side yang digunakan untuk mengembangkan aplikasi web. Laravel dibangun menggunakan PHP, sehingga memahami dasar-dasar PHP sangat penting untuk bekerja dengan Laravel.

b. Apache

Apache adalah salah satu web server paling populer yang digunakan untuk menjalankan aplikasi web berbasis PHP, termasuk aplikasi Laravel. Web server bertanggung jawab untuk menerima permintaan HTTP dari browser dan mengirimkan respons dari aplikasi yang berjalan di server.

c. **Laravel**

Laravel adalah framework PHP yang dirancang untuk membuat pengembangan web lebih cepat dan efisien. Laravel menawarkan sintaks yang elegan dan berbagai fitur bawaan yang mendukung pengembangan aplikasi berbasis MVC (Model-View-Controller).

d. **Laravel Passport**

Laravel Passport adalah paket yang disediakan oleh Laravel untuk mengimplementasikan otentikasi berbasis OAuth2. Passport memudahkan pengembangan API yang aman dengan memberikan cara standar untuk mengelola autentikasi token melalui OAuth2.

e. **Prettus Repository**

Prettus Repository adalah paket di Laravel yang mengimplementasikan Repository Pattern. Pattern ini membantu memisahkan logika akses data dari logika bisnis, memudahkan pengelolaan dan testing aplikasi.

f. **Swagger**

Swagger adalah alat yang digunakan untuk mendokumentasikan dan menguji API. Swagger memungkinkan pengembang untuk membuat deskripsi API yang dapat dibaca manusia dan mesin, serta menyediakan antarmuka interaktif untuk menguji API.

5.3. Frontend–Backend Communication

Gambaran Umum

Aplikasi ini dibangun menggunakan arsitektur client-server berbasis web, di mana:

- Frontend (client) berjalan di browser pengguna, dibangun dengan React dan ekosistemnya.
- Backend (server) dibangun dengan Laravel dan bertugas menangani logika bisnis, autentikasi, serta penyimpanan data di database.
- Komunikasi antara frontend dan backend dilakukan melalui RESTful API yang diamankan menggunakan OAuth2 (Laravel Passport).

Alur Komunikasi Antar Komponen

1. Frontend di-serve secara statis dibuka di browser pengguna.

2. Pengguna melakukan aksi seperti login, submit form, dsb.
3. Axios di frontend mengirim permintaan (request) HTTP ke backend (/api/...) menggunakan JWT token yang dikelola oleh Laravel Passport.
4. Backend (Laravel) memproses request:
 - Validasi input
 - Eksekusi logika bisnis
 - Ambil atau simpan data ke database melalui Repository Pattern
5. Response dikirim kembali ke frontend dalam format JSON.
6. Frontend memperbarui tampilan UI berdasarkan response menggunakan Redux state.

Tools dan Teknologi

Layer	Tool / Teknologi	Version	Fungsi
Frontend	React	18.3.1	Membangun antarmuka pengguna (UI) berbasis komponen.
	Redux/ Redux-Saga	5.0.1/1.3.0	Manajemen state global & efek samping (side effects) seperti API call.
	Axios	1.7.7	Melakukan request HTTP ke backend.
	Vite	5.4.8	Alat development untuk bundling & HMR.
	Tailwindcss/Bootstrap	3.4/4.5	Framework UI yang responsif dan siap pakai.
Backend	Laravel	11.0	Framework backend berbasis PHP dan arsitektur MVC.
	Laravel Passport	12.2	Autentikasi API.
	Prettus Repository	2.9	Implementasi Repository Pattern untuk layer akses data.
	Swagger(L5-Swagger)	8.6	Dokumentasi dan pengujian API secara interaktif.
Server/Hosting	Apache	2.4	Web server untuk melayani request.
Database	PostgreSQL	17.5	Penyimpanan data aplikasi.
Testing	Postman, Swagger UI	11.54/5.27	Pengujian endpoint dan autentikasi API.
Dokumentasi	Microsoft Office, PDF	2507	Pembuatan dokumentasi teknis proyek.

5.4. Database (Data Access Layer)

a. Fungsi : Menyimpan dan mengambil data.

b. Teknologi :

a. PostgreSQL

PostgreSQL adalah sistem manajemen basis data relasional open-source yang kuat dan kaya fitur. Dalam aplikasi Laravel, PostgreSQL berfungsi sebagai database tempat menyimpan dan mengelola data yang digunakan oleh aplikasi.

b. NAS

NAS adalah perangkat penyimpanan file yang terhubung ke jaringan dan memungkinkan berbagai pengguna serta aplikasi untuk mengakses file secara bersamaan. Dalam konteks aplikasi Laravel, NAS dapat digunakan untuk menyimpan file-file besar atau data statis yang diakses oleh aplikasi.

c. Integrasi Microsoft Sharepoint / One Drive

Microsoft SharePoint atau Microsoft OneDrive adalah aplikasi untuk kebutuhan penyimpanan dan sinkronisasi dokumen eksternal. Integrasi ini bertujuan untuk memudahkan akses dan kolaborasi terhadap dokumen-dokumen yang telah diunggah.

5.5. Email Notification

Fitur email notifikasi otomatis sebagai sarana pemberitahuan dari sistem yang dikirimkan ke alamat email pengguna. Notifikasi ini dapat berupa:

- Pengingat aktivitas,
- Informasi status,
- Tindakan yang perlu dilakukan pengguna berdasarkan kondisi atau jadwal tertentu.

5.6. Chat/Komunikasi

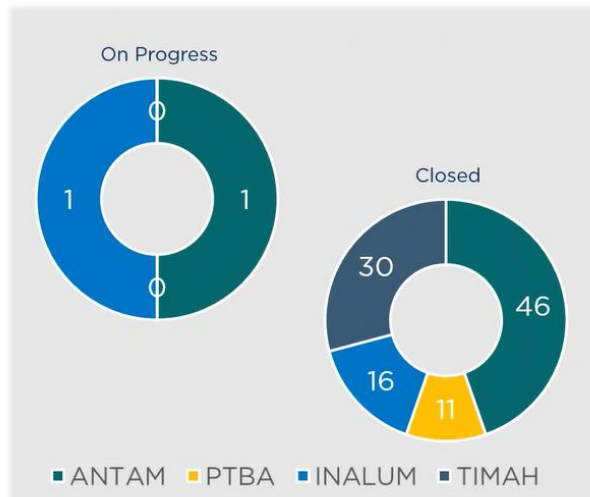
Fitur komunikasi dalam aplikasi berupa chat atau diskusi kontekstual yang memungkinkan pengguna untuk:

- Melakukan proses review dokumen atau arahan,
- Menambahkan tanggapan atau komentar secara langsung,

- Melakukan koordinasi terhadap perubahan status arahan secara efisien tanpa perlu berpindah platform.

5.7. Visualisasi & Reporting

Visualisasi data menggunakan Tableau atau aplikasi visualisasi sejenis yang bisa dilihat pada dashboard aplikasi.



Fitur ini memungkinkan pengguna:

- Melihat laporan berbasis data secara interaktif,
- Melakukan analisis mendalam terhadap data yang telah diinput,
- Mengekspor laporan ke dalam berbagai format seperti .PDF, dan .XLSX (Excel) untuk kebutuhan dokumentasi.

6. Security & Authorization

- a. Fungsi : Mengelola akses pengguna ke aplikasi.
- b. Teknologi :
 - Menggunakan JWT (JSON Web Token) pada backend Laravel memiliki beberapa manfaat penting, terutama dalam konteks otentikasi dan otorisasi di aplikasi berbasis API. JWT adalah standar terbuka yang memungkinkan informasi pengguna dikemas menjadi token yang ditandatangani, yang kemudian dapat digunakan untuk mengautentikasi permintaan API tanpa memerlukan sesi berbasis server.

- Semua endpoint API diamankan menggunakan middleware auth:api dari Laravel Passport.
 - Semua protokol komunikasi dari luar sistem ke dalam menggunakan SSL.
 - Data sensitif seperti kata sandi dienkripsi sebelum disimpan di database.
- Teknik pengamanan aplikasi dan data berdasarkan metode yang dikeluarkan oleh OWASP

7. Strategi Deployment dan Skalabilitas

7.1. Skalabilitas Horizontal

Aplikasi dapat ditingkatkan skalanya secara horizontal dengan menambahkan lebih banyak instance dari backend atau database. Load balancer akan mendistribusikan permintaan ke berbagai server yang didukung CI/CD untuk integrasi dan pengiriman yang cepat, sehingga menjamin ketersediaan (availability) dan performa.

7.2. Skalabilitas Vertikal

Skalabilitas vertikal dilakukan dengan meningkatkan kapasitas sumber daya pada satu server, seperti menambah jumlah CPU, RAM, penyimpanan, atau kecepatan I/O. Pendekatan ini memungkinkan aplikasi menangani beban kerja yang lebih besar tanpa mengubah arsitektur distribusi.

Beberapa strategi yang diterapkan dalam konteks ini antara lain:

- Peningkatan spesifikasi server backend, seperti upgrade instance di cloud provider (misalnya AWS EC2, Azure VM, atau GCP Compute Engine).
- Optimasi performa database dengan menggunakan penyimpanan SSD, memperbesar buffer size PostgreSQL, dan tuning query.
- Penggunaan caching layer (Redis/Memcached) untuk mengurangi beban komputasi dan akses ke database.
- Optimasi PHP-FPM dan web server (Apache/Nginx) dengan konfigurasi yang sesuai kebutuhan beban aplikasi.
- Konfigurasi worker queue (seperti Laravel Queue) untuk mempercepat proses asynchronous seperti pengiriman email, pemrosesan data, dll.

7.3. Strategi Caching

Caching digunakan untuk meningkatkan performa aplikasi dan mengurangi beban pada server backend serta database, dengan menyimpan data yang sering diakses agar tidak diproses atau diambil ulang dari sumber yang lebih lambat.

Berikut strategi caching yang digunakan pada aplikasi ini:

- Browser Cache: File statis (HTML, JS, CSS, gambar) dari hasil build React akan disimpan di browser menggunakan header Cache-Control dan ETag.
- State Caching: State yang sering digunakan misalnya data user atau preferensi UI disimpan di Redux store, dan dapat dipersist melalui redux-persist ke localStorage.
- Redis: sebagai cache store untuk menyimpan hasil query database yang sering diakses serta response dari API Eksternal.

8. Kesimpulan

Arsitektur aplikasi ini dirancang dengan fokus pada modularitas, efisiensi, dan skalabilitas. Setiap komponen dapat dikembangkan dan diuji secara terpisah, yang memungkinkan pengembangan aplikasi yang lebih mudah dan cepat.

< END OF DOCUMENT >